

Unit 1

Web Technology

What is Web?

Web (World Wide Web – WWW) is a collection of **web pages** that we access using a **browser** (like Chrome, Edge, Firefox).

- These web pages contain:
 - Text
 - Images
 - Videos
 - Links (hyperlinks)
- Each web page is identified by a **URL** (website address).

Important:

👉 Web is **not the internet**, it is a **service running on the internet**.

Example:

- www.google.com
 - www.amazon.in
 - www.youtube.com
-

What is Internet?

Internet is a **global network of computers and servers** connected to each other to share information.

- It is a **huge network of networks**
- Uses standard communication rules called **protocols**
- Internet supports many services, such as:

- Web (WWW)
- Email
- File transfer
- Video calls
- Online gaming

Important point:

👉 Internet is the **infrastructure**, web is just **one application** on it.

Example:

- When you send an email → Internet is used
- When you browse a website → Internet + Web both are used

Easy line to remember:

Internet = Road network

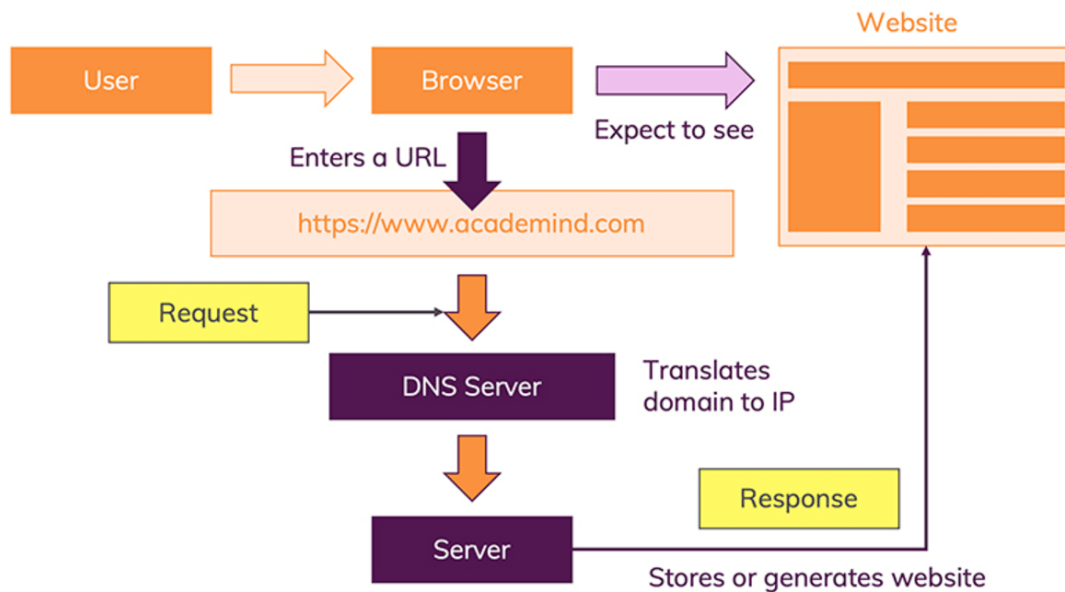
Web = Vehicles running on those roads

Difference between Web and Internet (Quick clarity)

Internet	Web
Global network	Service on internet
Exists since 1960s	Introduced in 1991
Supports many services	Only one service
Hardware + software	Web pages & websites

How does the Web work? (Overview)

Let's understand this **without technical overload**.



Step-by-step flow:

1. User types a **URL** in browser
👉 Example: `www.google.com`
2. Browser sends a **request** to the server
👉 “Please send me this website”
3. Server processes the request
👉 Finds required web page
4. Server sends **response** back
👉 HTML, CSS, images, etc.
5. Browser displays the web page on screen

📌 This model is called **Client–Server Architecture**

Roles:

- **Client** → Browser (Chrome, Edge)
- **Server** → Computer where website is stored

Real-life analogy:

Restaurant

- Customer = Client
 - Waiter = Internet
 - Kitchen = Server
 - Food = Web page
-

What is Web Technology?

Web Technology refers to the **tools, languages, protocols, and frameworks** used to **create, design, develop, and maintain websites and web applications**.

In simple words:

👉 Everything required to **build and run websites** is called web technology.

Web Technology includes:

- **Frontend technologies**
 - HTML (structure)
 - CSS (design)
 - JavaScript (behavior)
- **Backend technologies**
 - Server-side languages
 - Databases
- **Protocols**
 - HTTP / HTTPS
- **Browsers & Servers**

Important exam-friendly definition:

Web technology is a collection of software tools and techniques used to design, develop, and deploy web-based applications.

Real-Life Example (Complete picture)

Think of **Amazon website**:

- Internet → Connects your laptop to Amazon servers
- Web → Product pages you see
- Web Technology →
 - HTML → Product layout
 - CSS → Colors, fonts
 - JavaScript → Add to cart
 - Server → Order processing

One-line memory hook:

Internet connects, Web displays, Web Technology builds

▪ **Web Development Strategies**

Web Development Strategies are planned approaches, methods, and decisions followed while designing, developing, deploying, and maintaining a website or web application so that it:

- meets user requirements,
- performs efficiently,
- is secure,
- and is easy to maintain in the future.

Simple definition for students:

Web development strategies are systematic ways to plan and build websites efficiently and correctly.

Why “strategy” is needed?

Because building a website is **not just writing code**. It involves:

- understanding users,
 - selecting technologies,
 - designing layout,
 - ensuring performance & security.
-

Why are Web Development Strategies Important?

Without a proper strategy:

- Website may look good but **work poorly**
- Site may crash with more users
- Difficult to update or scale
- Security risks increase

With a proper strategy:

- Development becomes organized
- Time and cost are reduced
- Website becomes user-friendly and scalable

Real-life analogy:

Building a website without strategy is like constructing a building without a blueprint.

Major Web Development Strategies (Explained in Detail)

Now let's go **one by one**, clearly 🙋

◆ Understand Target Audience

What does it mean?

Before developing a website, developers must understand:

- Who will use the website?
- What is their age group?
- What are their needs and expectations?
- On which devices they will access the site?

Why is it important?

Different users have different requirements:

- Students → need simple UI
- Professionals → need fast & informative UI
- Elderly users → need readable fonts

Example:

- **E-learning website** → Simple navigation, readable content
- **Gaming website** → High graphics, animations

🧠 Analogy:

You don't design the same shoes for a child and an athlete.

◆ Define Clear Goals and Objectives

What does it mean?

Every website must have a **clear purpose**:

- Is it for information?

- Is it for selling products?
- Is it for user interaction?

Types of goals:

- Informational (college website)
- Transactional (shopping website)
- Social interaction (social media)

Why important?

Without clear goals:

- Website becomes confusing
- Features become unnecessary
- Users get frustrated

Example:

- Amazon → Goal: Sell products
- University website → Goal: Provide academic information

**Memory line:**

If the goal is unclear, the website will fail.

◆ Proper Planning and Requirement Analysis**What does it include?**

- Listing required features
- Understanding functional & non-functional needs
- Deciding number of pages
- Choosing technologies

Functional requirements:

- Login system
- Search feature

- Online payment

Non-functional requirements:

- Speed
- Security
- Reliability

Why important?

- Avoids rework
- Saves cost and time
- Reduces future errors

Analogy:

Measure twice, cut once.

◆ **Choose Appropriate Technology Stack**

What is a technology stack?

A **technology stack** is a combination of:

- Frontend technologies
- Backend technologies
- Database
- Server

Examples:

- Frontend → HTML, CSS, JavaScript
- Backend → Java, Node.js
- Database → MySQL, MongoDB

Why important?

Wrong technology choice may:

- Reduce performance
- Increase maintenance cost
- Limit scalability

Real-life example:

You can't cook every dish with the same utensils.

◆ **Responsive and User-Friendly Design**

What is responsive design?

Website should work properly on:

- Mobile
- Tablet
- Laptop
- Desktop

User-friendly design includes:

- Easy navigation
- Clear buttons
- Readable fonts
- Consistent layout

Why important?

- Most users access websites via mobile
- Poor UI leads to high bounce rate

Analogy:

A shop with poor layout loses customers.

◆ **Performance Optimization**

What does it mean?

Improving website speed and efficiency by:

- Compressing images
- Minimizing code
- Reducing server response time

Why important?

- Slow websites lose users
- Search engines penalize slow sites

Example:

- Google loads in milliseconds
- Heavy websites take long → users leave

Memory hook:

Faster website = happier users

◆ Security Considerations

What does it include?

- Protecting user data
- Preventing unauthorized access
- Using secure protocols

Security measures:

- HTTPS
- Authentication
- Authorization
- Input validation

Why important?

- Prevents data theft
- Builds user trust

Real-life analogy:

Locking your house before sleeping.

◆ **Testing and Maintenance**

Testing includes:

- Functional testing
- Performance testing
- Security testing

Maintenance includes:

- Bug fixing
- Feature updates
- Performance improvements

Why important?

- Websites are not “build once and forget”
- Continuous improvement is required

Analogy:

Just like vehicles need regular servicing.

• **History of Internet and Web**

(From Zero to Today's Era)

1 Era Before Internet (Before 1960)

Before the internet existed:

- Computers were **standalone machines**
- No communication between computers
- Data was shared using:

- Papers
- Magnetic tapes
- Physical storage

Problem at that time:

- Data sharing was slow
- No remote communication
- Computers were expensive and isolated

Analogy:

People had phones but no network to connect calls.

2 Birth of the Internet (1960s – 1970s)

Why Internet was needed?

During the **Cold War**, the US wanted:

- A communication system
- That would still work even if one part failed

ARPANET (1969)

- First computer network
- Developed by the US Department of Defense
- Connected universities and research centers

Key achievement:

- First message sent between two computers
- Message was “LOGIN” (system crashed after “LO” 😊)

Important point for exams:

ARPANET is the foundation of today's internet.

Development of Internet Protocols (1970s – 1980s)

◆ **TCP/IP (1983)**

- Standard communication rules were introduced
- Allowed different networks to communicate

 This is considered the **official birth of the Internet**

◆ **Email (1971)**

- First popular internet service
- Became widely used before the web

 **Analogy:**

Internet became a common language that all computers could understand.

Birth of the World Wide Web (1989 – 1991)

◆ **Why Web was created?**

Internet existed, but:

- It was complex
- Used only by scientists
- No user-friendly interface

◆ **World Wide Web (WWW)**

Invented by **Tim Berners-Lee** in 1989.

◆ **Key Contributions:**

- **HTML** → To create web pages
- **HTTP** → To transfer web pages
- **URL** → To locate web pages

 **Important clarification:**

Internet already existed

Web was built **on top of the internet**

Memory line:

Internet = Network

Web = Information layer

5 Early Web Era – Web 1.0 (1991 – 2000)

◆ **Characteristics:**

- Read-only web
- Static web pages
- No user interaction

◆ **Technologies used:**

- HTML
- Basic CSS
- No JavaScript initially

◆ **Examples:**

- Company information websites
- Online brochures

User role:

- Only read information
- No comments, no uploads

Analogy:

Like reading a newspaper — no interaction.

Interactive Web – Web 2.0 (2000 – 2010)

◆ Major change:

- Web became **interactive**
- Users could **create and share content**

◆ New features:

- Dynamic websites
- User accounts
- Social media
- Blogs and comments

◆ Technologies introduced:

- JavaScript
- AJAX
- CSS advancements
- Server-side scripting

◆ Examples:

- Facebook
- YouTube
- Wikipedia

User role:

- Read + Write content

Analogy:

From newspaper → to conversation.

7 Smart & Semantic Web – Web 3.0 (2010 – Present)

♦ What is Web 3.0?

Web that:

- Understands user behavior
- Provides personalized results
- Uses AI and machine learning

♦ Features:

- Smart search
- Voice assistants
- Recommendation systems
- Decentralization (Blockchain concept)

♦ Examples:

- Google personalized search
- Netflix recommendations
- Voice assistants

📌 User role:

- Read + Write + Smart interaction

🧠 Analogy:

Web that thinks for you.

8 Modern Internet Era (Today)

♦ Current Internet features:

- Cloud computing
- Mobile internet
- IoT (Internet of Things)
- 5G networks

◆ **Websites today:**

- Highly responsive
- Secure
- Fast
- Scalable

◆ **Web Applications:**

- Online banking
- E-commerce
- Video conferencing
- Online education

📌 **Important point:**

Today's web is application-centric, not page-centric.

Protocol	Full Form	Functionality (What it does)	Use Case Example
HTTP	HyperText Transfer Protocol	Defines how data (web pages, text, images) is transferred between a web browser and a web server using request-response model.	Opening a website like http://example.com
HTTPS	HyperText Transfer Protocol Secure	Secure version of HTTP. Uses SSL/TLS encryption to protect data, privacy, and authentication.	Online banking, shopping websites, Gmail
FTP	File Transfer Protocol	Used to upload and download files between client and server. Supports authentication using username and password.	Uploading website files to a hosting server using FileZilla
SFTP	Secure File Transfer Protocol	Secure version of FTP. Works over SSH and encrypts both data and commands.	Secure file sharing, server backups in companies
SMTP	Simple Mail Transfer Protocol	Used for sending emails from a client to a mail server or between mail servers.	Sending an email using Gmail or Outlook
IMAP	Internet Message Access Protocol	Allows users to read and manage emails directly from the mail server. Emails remain stored on the server.	Checking Gmail on mobile and laptop (emails stay synced)
POP3	Post Office Protocol (Version 3)	Downloads emails from server to local system and usually deletes them from server.	Downloading old emails in Outlook

Protocol	Full Form	Functionality (What it does)	Use Case Example
DNS	Domain Name System	Converts human-readable domain names into IP addresses that computers understand.	Typing <code>www.google.com</code> → DNS converts it to an IP address
TCP/IP	Transmission Control Protocol / Internet Protocol	TCP ensures reliable data delivery; IP handles addressing and routing of data packets.	Browsing websites, video calls, file downloads
SSH	Secure Shell	Provides secure remote login and command execution on another computer/server using encryption.	Admin logging into a server using <code>ssh user@ip</code>
SSL/TLS	Secure Sockets Layer / Transport Layer Security	Provides encryption and authentication for secure communication over networks.	Used behind HTTPS, online transactions, secure apps
MQTT	Message Queuing Telemetry Transport	Lightweight publish-subscribe protocol for IoT devices, designed for low bandwidth usage.	Smart home devices sending sensor data
WebSocket	—	Enables full-duplex (two-way) real-time communication between client and server.	Live chat apps, stock price updates, online games

Protocols Governing Web:

The Web and Internet don't run freely; they are governed by Protocols (set of rules). Protocols ensure that data is transmitted, received, and understood correctly between computers, browsers, and servers.

Here's a detailed explanation of the main protocols governing the web, with their functionality and real-world use cases 📖

1. HTTP (HyperText Transfer Protocol)

- **Functionality:**

HTTP defines how messages are formatted and transmitted between a **web browser** (client) and a **web server**. It's a **stateless protocol**, meaning every request is independent (server doesn't remember previous requests).

- **Use Case:**

- When you type `http://www.example.com`, your browser sends an HTTP request to fetch the page.
 - Used for **browsing websites, downloading files, API calls**.
-

2. HTTPS (HyperText Transfer Protocol Secure)

- **Functionality:**

It's HTTP + **SSL/TLS encryption**. Ensures that data transferred between browser and server is **encrypted**, safe from hackers.

- **Use Case:**

- Banking websites, e-commerce, Gmail, social media (Facebook, Instagram).
- Any website with login or payment gateway uses HTTPS (`https://`).

3. SMTP (Simple Mail Transfer Protocol)

- **Functionality:**

Used to **send emails** between mail servers and from email clients (like Outlook, Gmail) to a mail server.

- **Use Case:**

- Sending emails (e.g., Gmail sending your email to Yahoo).
 - Automated systems sending confirmation mails, OTPs.
-

4. IMAP (Internet Message Access Protocol)

- **Functionality:**

Allows users to **read and manage emails directly on the mail server**.
Keeps emails synchronized across devices.

- **Use Case:**

- If you open Gmail on mobile and laptop, IMAP ensures both show the same emails.
-

5. POP3 (Post Office Protocol version 3)

- **Functionality:**

Downloads emails from the server to your device, then usually deletes them from the server.

- **Use Case:**

- Old style of managing emails (when internet storage was limited).
- Still used in lightweight email clients for offline reading.

6. FTP (File Transfer Protocol)

- **Functionality:**
Used for **uploading and downloading files** between a client and server.
 - **Use Case:**
 - Developers upload website files to a hosting server.
 - Downloading large datasets from servers.
-

7. SFTP (Secure File Transfer Protocol)

- **Functionality:**
Same as FTP, but encrypted using **SSH (Secure Shell)** for secure file transfers.
 - **Use Case:**
 - Uploading confidential data to cloud servers.
 - Remote backup services.
-

8. DNS (Domain Name System Protocol)

- **Functionality:**
Converts **human-readable domain names** (like google.com) into **IP addresses** (like 142.250.190.14).
- **Use Case:**
 - When you type facebook.com, DNS finds its IP address and connects you to the correct server.

G. TCP/IP (Transmission Control Protocol / Internet Protocol)

- **Functionality:** Backbone of the Internet.
 - **IP:** Breaks data into packets and routes them.
 - **TCP:** Ensures all packets arrive correctly, in order.
 - **Use Case:**
 - Anytime you browse the web, send an email, or stream video, TCP/IP is working in the background.
-

10. UDP (User Datagram Protocol)

- **Functionality:**

Similar to TCP but **faster** and doesn't check packet order or errors. (No guarantee of delivery.)
 - **Use Case:**
 - Live video streaming (YouTube Live, Twitch).
 - Online gaming (Fortnite, PUBG).
 - VoIP (Zoom, WhatsApp calls).
-

11. Telnet

- **Functionality:**

Allows remote login into another computer/server using command-line interface.
- **Use Case:**
 - Earlier used for remote server management (now mostly replaced by SSH).

12. SSL/TLS (Secure Socket Layer / Transport Layer Security)

- **Functionality:**

Security protocol that encrypts communication between client and server (used inside HTTPS, FTPS, SMTPS, etc.).

- **Use Case:**

- Protects sensitive information (passwords, card details).
- Enables padlock 🔒 symbol in browsers.

In short analogy:

Think of **protocols as traffic rules of the Internet:**

- **HTTP/HTTPS** → Cars (websites) moving on the road.
- **SMTP/IMAP/POP3** → Postal system for emails.
- **FTP/SFTP** → Goods transport trucks for file transfer.
- **DNS** → Address book of the city.
- **TCP/IP s UDP** → Roads and lanes that ensure vehicles reach destinations.
- **SSL/TLS** → Seatbelts/helmets for safety.

Connecting to the Internet (Wired and Wireless)

To access websites, emails, cloud services, or web applications, a **device must connect to the Internet** using some communication medium and networking devices.

Internet connection happens mainly in **two ways**:

1. **Wired Internet Connection**
2. **Wireless Internet Connection**

Both methods require certain **networking devices**, each having a specific role.

Let's understand this step-by-step in a **clear, classroom-ready** manner.

1 Wired Internet Connection

In a wired connection, data travels through **physical cables** such as fiber, telephone lines, or Ethernet cables.

Basic Wired Connection Flow

Internet Service Provider (ISP)

→ Modem

→ Router/Switch

→ Computer or Network Devices

Wired connection is usually:

- Faster
- Stable
- Less interference
- Used in offices, labs, and servers

Devices Used in Wired Internet Connection

1. Modem (Modulator–Demodulator)

A **modem** connects your home or office network to the **Internet Service Provider (ISP)**.

Function:

- Converts digital signals from computer to analog signals for transmission.
- Converts incoming analog signals back to digital.

Simple Explanation:

Computers use **digital signals**, while internet lines may use **analog signals**. The modem converts between the two.

Example:

Fiber or broadband connection entering your home first goes to the modem.

 Analogy:

Modem is like a translator between your computer and internet provider.

2. Router

A **router** distributes the internet connection to multiple devices.

Function:

- Directs data packets to correct devices.
- Connects multiple devices to internet.
- Assigns IP addresses.

Example:

One router connects laptops, mobiles, TVs, etc., in a home.

 Analogy:

Router is like a traffic controller directing vehicles to correct roads.

3. Switch

A **network switch** connects multiple devices within the same network.

Function:

- Transfers data only to the intended device.
- Improves network efficiency.

Example:

Used in offices or computer labs to connect many PCs.

 Analogy:

Switch is like a smart postman delivering letters only to correct addresses.

4. Ethernet Cable

A physical cable connecting devices to network equipment.

Features:

- Fast and reliable
 - Used in offices, labs, servers
 - Less interference than wireless
-

2 Wireless Internet Connection

Wireless connection uses **radio waves** instead of cables.

Most homes and public places now use wireless internet.

Basic wireless flow:

ISP → Modem → Router/Wi-Fi Access Point → Wireless Devices

Devices Used in Wireless Internet

1. Wireless Router (Wi-Fi Router)

A wireless router sends internet signals using radio waves.

Function:

- Converts wired internet into Wi-Fi signals.
- Allows multiple wireless devices to connect.

Example:

Home Wi-Fi router connecting phones and laptops.

 Analogy:

Router is like a Wi-Fi tower inside your home.

2. Access Point (AP)

An **Access Point** expands wireless coverage.

Function:

- Extends network to larger areas.
- Used in offices, malls, campuses.

Example:

College campus Wi-Fi zones.

 Analogy:

Access Point acts like a signal booster tower.

3. Wireless Network Adapter

Device enabling computers to connect to Wi-Fi.

Forms:

- Built-in laptop Wi-Fi chip
- USB Wi-Fi adapters

3 Wired vs Wireless Connection

Feature	Wired Connection	Wireless Connection
Speed	Faster	Slightly slower
Stability	Very stable	Can fluctuate
Mobility	Limited	High mobility
Setup	Requires cables	Easy setup
Interference	Minimal	Possible interference

Real-Life Internet Connection Example

At home:

1. ISP cable enters house.
2. Connected to modem.
3. Modem connects to Wi-Fi router.
4. Router sends signals.
5. Devices connect via cable or Wi-Fi.

So:

- Laptop may use Wi-Fi.
- Desktop PC may use Ethernet cable.

5 Complete Analogy (Easy Memory Trick)

Think of internet like water supply:

- ISP → Water supply company
- Modem → Main valve
- Router → Water distributor
- Switch → Room pipelines
- Wi-Fi → Wireless water spray

Exam-Friendly Definition

Internet connection can be established using wired or wireless methods with devices such as modem, router, switch, access point, and network adapters to enable communication between user devices and the global internet.

Introduction to Internet Services and Tools

The Internet is not only used for browsing websites. It provides **many services and tools** that help users communicate, share information, and perform online activities.

To understand this topic, students must clearly know:

- What are Internet services?
 - What are Internet tools?
 - How they are used in daily life?
-

What are Internet Services?

Internet services are facilities or functions provided over the internet that allow users to communicate, access information, or perform tasks online.

Simple Definition

Internet services are services that allow users to exchange information and perform activities using the internet.

These services run on servers and are accessed through different applications or platforms.

What are Internet Tools?

Internet tools are software applications or programs that help users access and use internet services.

Simple Definition

Internet tools are programs used to access and utilize internet services.

For example:

- A browser is a tool used to access web pages.
 - Email apps are tools used to send emails.
-

Now we will study the **major Internet services and tools**, one by one.

Major Internet Services

◆ 1. World Wide Web (WWW)

The **World Wide Web (WWW)** is the most popular internet service.

What it does

- Allows users to access web pages and websites.
- Uses browsers to display information.

Example

Opening websites like:

- Google
- Amazon
- College websites

Used for

- Information searching
- Online shopping
- Learning
- Entertainment

Key concept

Web pages are connected through hyperlinks.

 Analogy:

WWW is like a huge online library of information.

◆ 2. Email Service

Email allows users to **send and receive messages electronically** over the internet.

Features

- Send messages instantly
- Attach files, images, documents
- Communication across the world

Examples

- Gmail
- Outlook
- Yahoo Mail

Uses

- Official communication
- Job applications
- Academic communication

 Analogy:

Email is a digital postal system.

◆ 3. File Transfer Service (FTP)

FTP (File Transfer Protocol) allows users to **upload and download files** between computers over the internet.

Uses

- Website file uploads
- Sharing large files
- Server data management

Example

Web developers upload website files to servers.

 Analogy:

FTP acts like a courier service for files.

◆ 4. Instant Messaging and Chat Services

Allows users to communicate instantly through text messages.

Features

- Real-time communication
- File and image sharing
- Voice notes

Examples

- WhatsApp Web
- Telegram
- Messenger

Uses

- Personal communication
- Office collaboration

 Analogy:

Chat services are like instant conversations online.

◆ 5. Voice and Video Calling (VoIP)

Voice over Internet Protocol (VoIP) enables voice and video communication over the internet.

Examples

- Zoom
- Google Meet
- Microsoft Teams

Uses

- Online meetings
- Online classes
- Video conferencing

 Analogy:

Like phone calls but through internet.

◆ 6. Online Services (E-commerce & Banking)

Internet allows online business and financial activities.

Examples

- Online shopping
- Online banking
- Ticket booking
- Food delivery

Uses

- Buying products
- Paying bills
- Booking services

◆ 7. Cloud Services

Cloud services allow storing and accessing data online instead of local storage.

Examples

- Google Drive
- OneDrive
- Dropbox

Uses

- File storage
- Backup
- Collaboration

 Analogy:

Cloud is like an online locker for files.

Internet Tools

Now let's see tools used to access these services.

◆ 1. Web Browser

A browser is software used to access websites.

Examples

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

Function

- Sends request to server
- Displays web pages

 Analogy:

Browser is a vehicle used to travel on the web.

◆ 2. Email Clients

Applications used to manage emails.

Examples

- Outlook
 - Thunderbird
 - Gmail web interface
-

◆ 3. FTP Tools

Used for file transfers between local computer and server.

Examples

- FileZilla
 - WinSCP
-

◆ 4. Communication Tools

Used for chat, voice, and video communication.

Examples

- Zoom
 - Skype
 - Teams
-

◆ 5. Search Engines

Tools used to find information on the web.

Examples

- Google
- Bing
- Yahoo

Function

- Searches billions of web pages
- Displays relevant results

 Analogy:

Search engine is like a librarian guiding you to books.

Real-Life Example (Complete Flow)

When a student submits an online assignment:

1. Uses browser (tool)
2. Accesses college website (WWW service)
3. Uploads file (FTP or web upload)
4. Receives email confirmation
5. Data stored in cloud

Multiple services and tools work together.

Exam-Friendly Conclusion

Internet services provide facilities like web browsing, email, file transfer, communication, and cloud storage, while internet tools such as browsers, email clients, and communication applications help users access and use these services efficiently.

Introduction to Client–Server Computing

Client–Server Computing is the **foundation of modern web applications**. Every time we open a website, send an email, or use an online app, a **client–server interaction** happens in the background.

Understanding this concept is very important for Web Technology students because **all web applications work on this model**.

1 What is Client–Server Computing? (Definition)

Client–Server computing is a network model in which:

- A **client** requests services or resources.
- A **server** provides those services or resources.

Both communicate over a network, usually the **Internet**.

Exam-Friendly Definition

Client–Server computing is a computing model in which multiple clients request services or data from a centralized server through a network.

2 Who is a Client?

A **client** is a device or program that:

- Sends requests
- Receives services or data from the server

Examples of clients

- Web browser (Chrome, Firefox)
- Mobile apps
- Laptop or mobile accessing a website

Example scenario

When a student opens a college website using Chrome, the browser acts as a **client**.

3 Who is a Server?

A **server** is a powerful computer or program that:

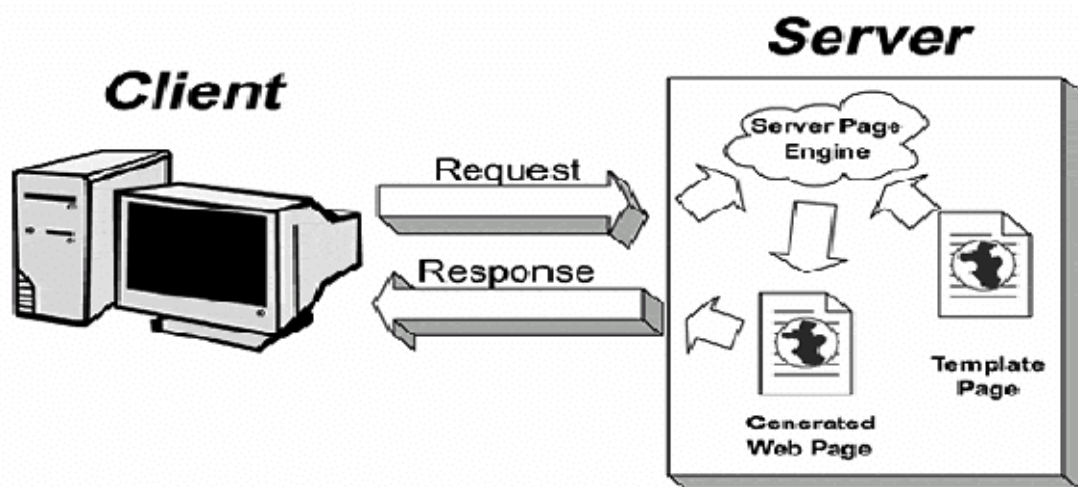
- Stores data or applications
- Processes client requests
- Sends responses back to clients

Examples of servers

- Website servers
- Database servers
- Email servers

For example, Google's servers send search results when users search something.

4 How Client-Server Computing Works (Overview)



Step-by-step working

1. User enters a website URL in browser.
2. Browser (client) sends request to server.
3. Server receives request and processes it.
4. Server prepares response (web page/data).

5. Response is sent back to client.

6. Browser displays the result.

This happens within milliseconds.

5 Example: Opening a Website

When a student opens **www.amazon.in**:

- Laptop/mobile → Client
- Amazon server → Server
- Internet → Communication medium
- Product page → Server response

So, client asks:

"Show me the Amazon homepage."

Server responds with webpage data.

6 Components of Client–Server Model

1. Client Device

User system that makes requests.

2. Server System

Stores and manages data/services.

3. Network

Medium connecting client and server.

4. Protocols

Rules of communication (HTTP, HTTPS).

7 Types of Client–Server Architecture

A. Two-Tier Architecture

Client directly communicates with server.

Example:

Client ↔ Database Server

Used in small systems.

B. Three-Tier Architecture (Common in Web Apps)

Consists of:

1. Client Layer (browser)
2. Application Server (logic processing)
3. Database Server (data storage)

Example:

Browser → Web Server → Database

This structure improves scalability and security.

8 Advantages of Client–Server Computing

✓ Centralized Data

Data stored at one place.

✓ Easy Maintenance

Server updates affect all clients.

✓ Better Security

Server controls access.

✓ Resource Sharing

Multiple users access services.

9 Disadvantages

✗ Server Failure

If server stops, services stop.

✗ Network Dependency

Requires stable internet.

✗ High Server Cost

Servers require maintenance.

10 Real-Life Analogy

Think of a **restaurant**:

- Customer → Client
- Waiter → Network
- Kitchen → Server
- Food → Response

Customer orders food → Kitchen prepares → Waiter delivers.

1 1 Real-Life Example in Daily Life

When using:

- Google Search
- Online Banking
- Netflix
- College Portal

Client–server interaction happens every time.

Exam-Friendly Conclusion

Client–Server computing is a communication model where clients request services and centralized servers process and provide the requested resources over a network.

Introduction to HTML

HTML is the **foundation of every website**. Before learning CSS, JavaScript, or backend technologies, students must clearly understand HTML because **all web pages start with HTML**.

1 What is HTML?

HTML stands for **HyperText Markup Language**.

It is the standard language used to **create and structure web pages**.

Definition (Exam-Friendly)

HTML is a markup language used to create and structure content on web pages using tags and elements.

Breaking the term HTML

HyperText

- Text that contains links to other pages.
- Clicking links allows navigation between pages.

Markup

- Means adding tags to structure content.

Language

- A standard set of rules used to create web documents.

What HTML Actually Does

HTML defines:

- Page structure, headings, paragraphs, images, Links, Tables, Forms, Lists, etc.

HTML does **not** handle design or logic; it only structures content.

Real-Life Analogy

Think of a house:

- HTML → Structure of house (walls, rooms)
- CSS → Interior design
- JavaScript → Electrical and automation system

So,

HTML builds the skeleton of a webpage.

2 Features of HTML

Now let us see important features students should remember.

1. Simple and Easy to Learn

HTML uses simple tags and syntax.

Example:

```
<p>Hello</p>
```

Even beginners can learn it quickly.

2. Platform Independent

HTML works on all operating systems:

- Windows
- Linux
- Mac
- Mobile devices

Any browser can display HTML pages.

3. Tag-Based Language

HTML uses **tags** to define elements.

Example:

```
<h1>Heading</h1>
```

Tags tell browser how to display content.

4. Supports Multimedia

HTML allows adding:

- Images
- Audio
- Video
- Animations

Example:

```

```

5. Hyperlink Support

HTML allows linking pages.

Example:

```
<a href="page2.html">Go to Page 2</a>
```

6. Integrates with CSS & JavaScript

HTML works together with:

- CSS for design
 - JavaScript for interaction
-

7. Open Standard

HTML is free to use and supported worldwide.

8. Supports Forms and User Input

HTML allows user interaction via forms.

Example:

- Login forms
- Registration forms
- Search boxes

3 Basic Structure of an HTML Document

Every HTML page follows a standard structure.

Basic HTML Template

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>My First Web Page</title>
```

```
</head>
```

```
<body>
```

```
  <h1>Welcome to HTML</h1>
```

```
  <p>This is my first webpage.</p>
```

```
</body>
```

```
</html>
```

Explanation of Each Line

Line 1: DOCTYPE Declaration

`<!DOCTYPE html>`

This tells the browser:

- The document is written in **HTML5**.
- Browser should render page correctly.

Think of it as telling browser:

"This is an HTML document."

Line 2: Opening HTML Tag

`<html>`

This is the root element of the page.

Everything inside the webpage comes within this tag.

Line Last: Closing HTML Tag

`</html>`

Marks end of HTML document.

Head Section

`<head>`

Contains **information about webpage**, not visible content.

Used for:

- Page title
- Meta data
- CSS links

- Scripts

Closing:

</head>

Title Tag

<title>My First Web Page</title>

This text appears:

- On browser tab
- As page title

Example:

Tab shows → *My First Web Page*

Body Section

<body>

Contains all visible content:

- Text
- Images
- Links
- Videos
- Tables

Everything users see goes here.

Closing:

</body>

Heading Tag

<h1>Welcome to HTML</h1>

Displays main heading.

HTML supports headings:

<h1> to <h6>

where:

- h1 = largest
 - h6 = smallest
-

Paragraph Tag

<p>This is my first webpage.</p>

Defines paragraph text.

5 Complete Flow Understanding

When browser loads page:

1. Reads DOCTYPE
 2. Reads HTML structure
 3. Loads head info
 4. Displays body content
-

6 Real-Life Analogy

Think of HTML page like a book:

- <html> → Book cover
 - <head> → Book information
 - <title> → Book title
 - <body> → Book content
 - <h1> → Chapter title
 - <p> → Paragraph text
-

7 Memory Tip for Students

Remember structure as:

DOCTYPE

HTML

HEAD

TITLE

BODY

CONTENT

Exam-Friendly Conclusion

HTML is the standard markup language used to create and structure web pages using tags and elements. Every HTML document follows a basic structure containing doctype declaration, html, head, title, and body elements.

HTML Tags and Elements

In HTML, **tags and elements** are the basic building blocks used to create web pages. Every webpage is made using multiple HTML elements formed by tags.

Here we will understand the concept **briefly and clearly**, without going into too much detail.

1 What are HTML Tags?

HTML tags are keywords enclosed within angle brackets < > that tell the browser how content should appear on a webpage.

Example

<p>

This is a paragraph tag.

Tags usually come in pairs:

- Opening tag
- Closing tag

Example:

```
<p>This is a paragraph</p>
```

Here:

- `<p>` → Opening tag
- `</p>` → Closing tag

The closing tag contains a **forward slash**.

Some Common HTML Tags	
Tag	Purpose
<code><h1></code>	Heading
<code><p></code>	Paragraph
<code><a></code>	Link
<code></code>	Image
<code><table></code>	Table
<code><form></code>	Form

2 What is an HTML Element?

An **HTML element** consists of:

- Opening tag
- Content
- Closing tag

Example

```
<p>This is HTML.</p>
```

The entire line forms an **HTML element**.

So,

Element = Opening Tag + Content + Closing Tag

3 Difference Between Tag and Element	
Tag	Element
Only the keyword in brackets	Complete structure
Example: <code><p></code>	Example: <code><p>Hello</p></code>
Instructions	Content + instruction

4 Empty Elements

Some elements do not have closing tags because they contain no content.

Example:

`<br>`

``

These are called **empty elements**.

5 Simple Memory Trick

Remember:

Tag is a label, element is the complete content block.

Exam-Friendly Definition

HTML tags are markup keywords enclosed in angle brackets used to define webpage structure, while HTML elements consist of opening tag, content, and closing tag forming complete webpage components.

HTML Lists

HTML lists are used to **display items in an organized manner** on a webpage. Lists help present information clearly, especially when showing items, steps, or categories.

HTML provides **three types of lists**.

1 What is an HTML List?

An **HTML list** is used to group related items together in a structured format using list tags.

Example Uses

- Showing steps in a process
 - Displaying menu items
 - Listing features or products
 - Navigation menus
-

2 Types of HTML Lists

There are **three types of lists** in HTML:

1. Ordered List
 2. Unordered List
 3. Description List
-

◆ 1. Ordered List (Numbered List)

An **ordered list** displays items in a **numbered sequence**.

Tag Used

`<ol>`

Each item is written using:

`<li>`

Example

```
<ol>

  <li>Wake up</li>

  <li>Get ready</li>

  <li>Go to college</li>

</ol>
```

Output Appears As

1. Wake up
2. Get ready
3. Go to college

Where Used

- Step-by-step instructions
 - Ranking lists
 - Procedures
-

◆ 2. Unordered List (Bullet List)

An **unordered list** shows items using **bullet points**.

Tag Used

```
<ul>
```

Each item again uses:

```
<li>
```

Example

```
<ul>

  <li>Apple</li>

  <li>Mango</li>

  <li>Banana</li>

</ul>
```

Output Appears As

- Apple
- Mango
- Banana

Where Used

- Product features
 - Menu items
 - Category lists
-

◆ 3. Description List

A **description list** displays terms and their descriptions.

Tags Used

- `<dl>` → Description list
- `<dt>` → Term name
- `<dd>` → Description

Example

```
<dl>
```

```
<dt>HTML</dt>
```

```
<dd>Language used to create web pages.</dd>
```

```
<dt>CSS</dt>
```

```
<dd>Used for webpage styling.</dd>
```

```
</dl>
```

Used For

- Definitions
- Glossaries
- FAQ sections

Exam-Friendly Conclusion

HTML lists are used to organize related items in webpages and are of three types: ordered lists, unordered lists, and description lists.

HTML Tables

HTML tables are used to **display data in rows and columns** on a webpage, similar to tables in books or spreadsheets.

Tables are commonly used to display:

- Student records
- Product lists
- Timetables
- Reports
- Employee data

1 What is an HTML Table?

An **HTML table** is created using table-related tags to organize data into **rows and columns**.

Exam Definition

An HTML table is used to display structured data in tabular form using rows and columns.

2 Basic Structure of a Table

A table consists of:

- Table container
- Rows
- Columns (cells)
- Headers

Basic flow:

```
<table>

  <tr>

    <th>Header</th>

    <th>Header</th>

  </tr>


  <tr>

    <td>Data</td>

    <td>Data</td>

  </tr>

</table>
```

3 Tags Used in HTML Tables

Now we will explain **all important table tags**.

◆ 1. <table> Tag

Defines the table.

Example:

```
<table>
```

All table content comes inside this tag.

◆ 2. <tr> — Table Row

Defines a row in the table.

Example:

```
<tr>
```

Each row is created using <tr>.

◆ 3. <th> — Table Header

Defines header cells.

- Header text appears **bold and centered** by default.

Example:

```
<th>Name</th>
```

◆ 4. <td> — Table Data

Defines normal data cells.

Example:

```
<td>Anshika</td>
```

◆ 5. <caption> — Table Title

Adds a title to the table.

Example:

```
<caption>Student Data</caption>
```

◆ 6. <thead> — Table Head Section

Groups header content.

◆ 7. <tbody> — Table Body Section

Groups main data content.

◆ 8. <tfoot> — Table Footer Section

Used for totals or summary.

◆ 9. <colgroup> and <col>

Used to apply formatting to columns.

4 Important Table Attributes (Commonly Used)	
Attribute	Purpose
border	Adds table border
rowspan	Merge rows
colspan	Merge columns
width	Table width
align	Alignment

5 Perfect Example Using All Table Tags

Below example demonstrates nearly all major table tags.

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Student Marks Table</title>
```

```
</head>
```

```
<body>
```

```
<table border="1" width="60%">
```

```
<caption><b>Student Marks Report</b></caption>
```

```
<tr>
```

```
<th rowspan="2">Roll No</th>
```

```
<th colspan="3">Marks</th>
```

```
</tr>
```

```
<tr>
  <th>Math</th>
  <th>Science</th>
  <th>English</th>
</tr>
```

```
<tr>
  <td>101</td>
  <td>85</td>
  <td>90</td>
  <td>88</td>
</tr>
```

```
<tr>
  <td>102</td>
  <td>78</td>
  <td>82</td>
  <td>80</td>
</tr>
```

```
<tr>
  <td>103</td>
  <td>92</td>
  <td>89</td>
  <td>94</td>
</tr>
```

```

<tr>
  <td colspan="4" align="center">
    Overall Result: Good Performance
  </td>
</tr>
</table>
</body>
</html>

```

Output:

Student Marks Report			
Roll No	Marks		
	Math	Science	English
101	85	90	88
102	78	82	80
103	92	89	94
Overall Result: Good Performance			

Frames in HTML

Frames in HTML were used to **divide a browser window into multiple sections**, where each section could display a different webpage.

This allowed developers to show **multiple pages in one browser window simultaneously**.

1 What are Frames in HTML?

A **frame** is a section of the browser window that displays a separate HTML document.

Instead of loading one webpage, the screen is split into parts, and each part loads different content.

Exam Definition

Frames in HTML are used to divide a browser window into multiple sections, each capable of displaying a separate HTML document.

2 Why Frames Were Used?

Frames were mainly used for:

- Navigation menus
- Header and content separation
- Sidebars
- Displaying multiple documents at once

Example:

- Left side: Menu
 - Right side: Content
-

3 Important Note (Modern Usage)

Frames using `<frameset>` are **obsolete in HTML5** and are no longer recommended.

Modern websites use:

- CSS layouts
- `<iframe>`
- Flexbox/Grid

However, frames are still studied for **academic understanding**.

4 Tags Used in Frames

Tag	Purpose
<code><frameset></code>	Defines frame layout
<code><frame></code>	Defines individual frame
<code><noframes></code>	Content for unsupported browsers
<code><iframe></code>	Inline frame inside webpage

5 Types of Frames in HTML

Frames are mainly divided based on layout.

Types:

1. Row Frames
2. Column Frames
3. Mixed Frames
4. Inline Frames

◆ 1. Row Frames

Browser window is divided **horizontally**.

Example

```
html

<!DOCTYPE html>
<html>
<frameset rows="30%,70%">
  <frame src="top.html">
  <frame src="bottom.html">
</frameset>
</html>
```

Window splits into:

- Top frame (30%)
- Bottom frame (70%)

Used for header + content layout.

◆ 2. Column Frames

Window is divided **vertically**.

Example

```
<!DOCTYPE html>
<html>
<frameset cols="30%,70%">
  <frame src="menu.html">
  <frame src="content.html">
</frameset>
</html>
```

Window splits into:

- Left menu
- Right content

Common in old websites.

◆ 3. Mixed Frames

Combination of rows and columns.

```
<!DOCTYPE html>
<html>
<frameset rows="30%,70%">
  <frame src="header.html">

  <frameset cols="30%,70%">
    <frame src="menu.html">
    <frame src="content.html">
  </frameset>
</frameset>
</html>
```

Layout:

- Header at top
- Bottom split into menu & content

◆ 4. Inline Frames (iframe)

Inline frames display another webpage **inside the current page**.

Unlike frameset, it works within normal HTML pages.

Example

```
<!DOCTYPE html>
<html>
<body>

<h2>Google Page</h2>

<iframe src="https://www.example.com"
        width="600"
        height="300">
</iframe>

</body>
</html>
```

Used today for:

- Embedding videos
- Maps
- External content

6 Advantages of Frames

- Multiple pages visible at once
- Reusable navigation sections
- Easy layout separation

7 Disadvantages of Frames

- Poor SEO support
- Bookmarking issues
- Navigation problems
- Not mobile-friendly

8 Real-Life Analogy

Think of a TV news channel:

- Top section → Headlines
- Side section → News list
- Main section → Live video

Each part shows different content simultaneously.

HTML Forms

HTML forms are one of the most important parts of web development because they allow **users to enter and submit data** to a server.

Forms are used in almost every website where user interaction is required.

Examples include:

- Login pages
- Registration forms
- Feedback forms
- Online applications
- Payment forms

1 What is an HTML Form?

An **HTML form** is used to collect user input and send it to a server for processing.

Exam-Friendly Definition

An HTML form is used to collect user input and submit it to a server using various input controls.

2 Why Forms are Needed?

Forms allow users to:

- Enter personal details
- Login to accounts
- Search information
- Submit feedback
- Upload files

Without forms, websites would only display information, not collect it.

3 Basic Structure of a Form

Example:

```
<form>
```

Input fields go here

```
</form>
```

All input elements are placed inside the `<form>` tag.

4 Important Attributes of `<form>` Tag

◆ action

Specifies where form data is sent.

Example:

```
<form action="submit.php">
```

◆ method

Specifies how data is sent.

Two common methods:

- GET → data visible in URL
- POST → data hidden

Example:

```
<form method="post">
```

5 Important Tags Used in Forms

Now we study the **important form tags**.

◆ 1. <input> Tag

Most commonly used form element.

Used to create different input controls.

Common Input Types	
Type	Purpose
text	Text input
password	Hidden text
email	Email input
number	Numeric input
radio	Single selection
checkbox	Multiple selection
submit	Submit form
reset	Reset fields
file	Upload file
date	Date selection

Example

```
<input type="text" name="username">
```

◆ 2. <label> Tag

Defines label for input field.

Example:

```
<label>Name:</label>
```

Improves accessibility.

◆ 3. **<textarea> Tag**

Used for multi-line text input.

Example:

```
<textarea rows="4" cols="30"></textarea>
```

Used in feedback forms.

◆ 4. **<select> Tag**

Creates dropdown list.

Example:

```
<select>
```

```
  <option>India</option>
```

```
  <option>USA</option>
```

```
</select>
```

◆ 5. **<option> Tag**

Defines items inside dropdown.

◆ 6. **<button> Tag**

Creates clickable button.

Example:

```
<button>Submit</button>
```

◆ 7. <fieldset> Tag

Groups related form elements.

◆ 8. <legend> Tag

Provides title for fieldset.

Complete Example:

```
<!DOCTYPE html>

<html>
<head>
<title>Registration Form</title>
</head>
<body>

<h2>Student Registration Form</h2>
<form action="submit.php" method="post">
<label>Name:</label>
<input type="text" name="name"><br><br>

<label>Password:</label>
<input type="password" name="pass"><br><br>

<label>Gender:</label>
<input type="radio" name="gender"> Male
<input type="radio" name="gender"> Female<br><br>
```

```
<label>Hobbies:</label>
<input type="checkbox"> Reading
<input type="checkbox"> Sports<br><br>

<label>Country:</label>
<select>
<option>India</option>
<option>USA</option>
<option>UK</option>
</select><br><br>

<label>Address:</label><br>
<textarea rows="4" cols="30"></textarea><br><br>

<input type="submit" value="Register">
<input type="reset" value="Clear">
</form>
</body>
</html>
```

Output:

Student Registration Form

Name:

Password:

Gender: ☐ Male ☐ Female

Hobbies: ☐ Reading ☐ Sports

Country:

Address:

XML (Extensible Markup Language)

XML is widely used in web technologies for **storing and transporting data** between systems.

Unlike HTML, which displays data, XML is mainly used to **store and organize data** in a structured format.

1 What is XML?

XML stands for **Extensible Markup Language**.

It is a markup language used to **store, organize, and transport data** in a structured and readable format.

Exam-Friendly Definition

XML is a markup language designed to store and transport data in a structured, self-descriptive format.

2 Why XML is Used?

XML is used because:

- It stores data in structured form.
- It is readable by humans and machines.
- It allows data exchange between different systems.
- It is platform independent.

Example uses:

- Data exchange between applications
 - Web services
 - Configuration files
 - Database data transfer
-

3 Example of XML Document

```
<?xml version="1.0"?>
```

```
<student>
```

```
  <name>Anshika</name>
```

```
  <branch>CSE</branch>
```

```
  <year>3</year>
```

```
</student>
```

Explanation

- <student> is root element.
 - <name>, <branch>, <year> store data.
 - Tags describe the data.
-

4 Features of XML

- User-defined tags allowed
 - Platform independent
 - Self-descriptive data
 - Supports hierarchical structure
 - Easy data sharing
-

5 What is XML Validation?

XML validation checks whether an XML document follows a **defined structure and rules**.

A validated XML document ensures:

- Correct structure
 - Correct tag usage
 - Correct data format
-

Two Important Terms

Well-Formed XML

XML is well-formed if:

- Proper opening and closing tags exist
- Tags are properly nested
- Case sensitivity is maintained
- One root element exists

Example error:

```
<name>Rahul</Name>
```

(Tag mismatch)

Valid XML

An XML document is valid if it:

- Is well-formed
- Follows rules defined in DTD or XML Schema.

Types of XML Validation

XML validation mainly occurs using two methods:

1. DTD (Document Type Definition)
2. XML Schema (XSD)

◆ 1. DTD Validation

DTD defines rules and structure of XML document.

It specifies:

- Allowed elements
- Element order
- Child elements
- Attributes

Example DTD

```
xml

<!DOCTYPE student [
<!ELEMENT student (name, branch, year)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT branch (#PCDATA)>
<!ELEMENT year (#PCDATA)>
]>
```

DTD ensures XML follows defined structure.

Advantages

- Simple
- Easy to write

Limitations

- No strong data type support
 - Limited validation capability
-

◆ 2. XML Schema (XSD) Validation

XML Schema is a more powerful way to validate XML.

It defines:

- Elements
- Attributes
- Data types
- Value restrictions

Example XML Schema

```
xml
<xs:element name="year" type="xs:integer"/>
```

This ensures year must be a number.

Advantages

- Supports data types
- More powerful validation
- Written in XML format
- Better control over structure

7 Difference Between DTD and XML Schema

Feature	DTD	XML Schema
Syntax	Non-XML	XML-based
Data Types	Not supported	Supported
Flexibility	Limited	High
Namespace Support	No	Yes

Types of DTD in XML (With Examples)

DTD (Document Type Definition) is used to **define the structure and legal elements of an XML document**.

DTD tells what elements and structure are allowed in XML.

DTD can be included in XML documents in **two ways**, known as **types of DTD**.

1 What is DTD?

DTD defines:

- Allowed elements
- Order of elements
- Attributes
- Structure of XML document

It ensures XML data follows a predefined structure.

2 Types of DTD

There are **two types of DTD**:

1. Internal DTD
2. External DTD

◆ **1. Internal DTD:** An **Internal DTD** is written **inside the XML document itself**.

The DTD rules are included within the XML file using the `<!DOCTYPE>` declaration.

Structure of Internal DTD

DTD rules are written between:

```
<!DOCTYPE root-element [  
    DTD rules  

```

Example: Internal DTD

XML Document with Internal DTD

```
xml  
  
<?xml version="1.0"?>  
  
<!DOCTYPE student [  
    <!ELEMENT student (name, branch, year)>  
    <!ELEMENT name (#PCDATA)>  
    <!ELEMENT branch (#PCDATA)>  
    <!ELEMENT year (#PCDATA)>  
]>  
  
<student>  
    <name>Anshika</name>  
    <branch>CSE</branch>  
    <year>3</year>  
</student>
```

Explanation

DTD rules are written inside XML:

- student contains name, branch, year
- Elements contain parsed character data.

Advantages

- Easy for small documents.
 - No separate file required.
-

Limitation

- Difficult to reuse in multiple XML files.
-

◆ **2. External DTD:** An **External DTD** is stored in a **separate file** and linked to XML document. DTD rules are reused across multiple XML files.

Structure

XML file references external DTD file:

```
<!DOCTYPE root-element SYSTEM "filename.dtd">
```

Example: External DTD

Step 1: Create DTD file (student.dtd)

```
xml

<!ELEMENT student (name, branch, year)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT branch (#PCDATA)>
<!ELEMENT year (#PCDATA)>
```

Step 2: XML file linking DTD

```
xml

<?xml version="1.0"?>

<!DOCTYPE student SYSTEM "student.dtd">

<student>
  <name>Anshika</name>
  <branch>CSE</branch>
  <year>3</year>
</student>
```

Explanation

DTD is stored separately and referenced using SYSTEM keyword.

Advantages

- Reusable across many XML files.
- Easy maintenance.
- Suitable for large projects.

3 Difference Between Internal and External DTD		
Feature	Internal DTD	External DTD
Location	Inside XML file	Separate file
Reusability	Limited	High
Maintenance	Harder	Easier
File Requirement	Single file	Two files

XSD (XML Schema Definition)

XSD is used to **define the structure and rules of an XML document**. It is more powerful and flexible than DTD and is commonly used in modern applications.

1 What is XSD?

XSD (XML Schema Definition) is a language used to describe the **structure, elements, attributes, and data types** of an XML document.

Exam-Friendly Definition

XSD is used to define the structure and data types of elements and attributes in an XML document.

2 Why XSD is Needed?

DTD has some limitations:

- No proper data type support
- Limited validation capability
- Not written in XML syntax

XSD overcomes these problems by:

- Supporting data types
 - Allowing value restrictions
 - Supporting namespaces
 - Being written in XML format
-

3 Simple Example of XSD

XML File

```
xml

<student>
  <name>Anshika</name>
  <age>21</age>
</student>
```

XSD File

```
xml

<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <xs:element name="student">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="name" type="xs:string"/>
        <xs:element name="age" type="xs:integer"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

</xs:schema>
```



Here:

- Name must be text.
 - Age must be number.
-

Types of XSD

XSD elements are mainly classified into **two types**:

1. Simple Type
 2. Complex Type
-

◆ 1. Simple Type in XSD

Definition

A **Simple Type** element:

- Contains only text.
- Cannot contain child elements.
- Cannot have attributes.

Example

XML:

```
xml
<age>21</age>
```

XSD:

```
xml
<xs:element name="age" type="xs:integer"/>
```

This ensures:

- Age must be a number.

◆ 2. Complex Type in XSD

Definition

A **Complex Type** element:

- Contains other elements
- Can contain attributes
- Can have structured data

Example XML

```
xml

<student>
  <name>Anshika</name>
  <age>21</age>
</student>
```

XSD

```
xml

<xs:element name="student">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="name" type="xs:string"/>
      <xs:element name="age" type="xs:integer"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Here:

- Student contains multiple elements.

5 Difference Between Simple and Complex Types

Feature	Simple Type	Complex Type
Contains child elements	No	Yes
Contains attributes	No	Yes
Structure	Single value	Structured data

DOM (Document Object Model)

DOM is a very important concept in web technology because it allows **programs and scripts (like JavaScript) to access and modify web pages dynamically.**

Whenever a webpage loads in a browser, the browser creates a **DOM structure** for that page.

1 What is DOM?

DOM (Document Object Model) is a programming interface that represents an HTML or XML document as a structured tree of objects.

Exam-Friendly Definition

DOM is a tree-like structure representing an HTML or XML document, allowing programs to access and modify page content, structure, and styles.

2 Why DOM is Needed?

Without DOM:

- Web pages would be static.
- Content could not change dynamically.

With DOM:

- Content can be changed dynamically.
- Elements can be added or removed.
- Styles can be modified.
- User interactions can update pages.

Example:

- Form validation
- Dynamic content loading
- Button click updates text

3 How DOM Works

When a webpage loads:

1. Browser reads HTML.
2. Converts HTML into DOM tree.
3. Each element becomes an object (node).
4. Scripts can access and modify these objects.

So browser converts webpage into a **programming structure**.

4 DOM Structure (Tree Representation)

Example HTML:

```
<html>
<body>
<h1>Hello</h1>
<p>Welcome</p>
</body>
</html>
```

DOM Tree Structure:

Document

```
|
html
|
body
|---- h1
|   |
|   Text
```

```
|
|---- p
|
Text
```

This hierarchical structure is called DOM Tree.

5 DOM Terminology

Document

Entire webpage.

Node

Each element or content in document.

Element Node

HTML tags like <p>, <div>.

Text Node

Text inside elements.

Attribute Node

Attributes of elements.

6 Types of DOM

DOM is used with:

1. HTML DOM
 2. XML DOM
-

HTML DOM

Used for HTML documents.

Example:
Changing webpage content using JavaScript.

XML DOM

Used to access XML data structure.

Used in data exchange applications.

7 DOM Operations (Basic Idea)

Using DOM, scripts can:

- Access elements
- Change content
- Modify styles
- Create elements
- Delete elements

Example idea:
Button click changes heading text.

8 Real-Life Analogy

Think of DOM like a **family tree**:

- Grandparent → Root node
- Parent → Child node
- Children → Sub-elements

Each member is connected hierarchically.

9 Memory Trick

Remember:

HTML page → Converted into → DOM tree

SAX (Simple API for XML)

SAX is a method used to **read and process XML documents** efficiently, especially when dealing with **large XML files**.

It is one of the techniques used to parse XML documents.

1 What is SAX?

SAX stands for **Simple API for XML**.

It is an **event-based XML parser** used to read XML documents sequentially.

Exam-Friendly Definition

SAX is an event-driven XML parsing technique that reads XML documents sequentially without loading the entire document into memory.

2 Why SAX Parser is Needed?

When XML files are very large:

- Loading the entire file into memory becomes slow.
- System memory usage increases.

SAX solves this by:

- Reading data step-by-step.
 - Processing data as it appears.
 - Not storing the whole document.
-

3 How SAX Works

SAX parser reads XML document **line by line** and generates events.

Typical events:

- Start of document
- Start of element

- Element content
- End of element
- End of document

Program reacts to these events.

Example XML File

```
<student>  
  <name>Anshika</name>  
  <branch>CSE</branch>  
</student>
```

SAX Processing Flow

While reading:

1. Start document event
2. Start element <student>
3. Start element <name>
4. Read text "Anshika"
5. End element <name>
6. Start element <branch>
7. Read text "CSE"
8. End element <branch>
9. End element <student>

Parser moves forward only.

Characteristics of SAX Parser

- Event-driven

- Sequential reading
 - Low memory usage
 - Faster for large files
 - Cannot move backward
-

6 Advantages of SAX

- ✓ Low memory consumption
 - ✓ Fast processing
 - ✓ Suitable for large XML files
-

7 Disadvantages of SAX

- ✗ Cannot access data randomly
- ✗ No document structure stored
- ✗ Harder programming logic